演算法分析與正確性

把程式跑完，只回答了「這次發生什麼」。分析演算法還要回答：**所有合法輸入都會得到正確結果嗎？輸入變大時，成本如何增加？**

本章依序建立規格與證明、逐步計數、漸近比較、級數與迴圈分析。插入排序會作為共同案例，從每次搬移一路走到一般成本公式。

先備知識：陣列、迴圈、基本代數與對數；面積與極限推導另用到基礎微積分。

先問的問題	需要交代的證據
什麼才算正確答案？	輸入條件、輸出條件與邊界
為什麼每次都正確？	不變量與終止性
到底花多少成本？	成本模型、計數對象、一般界限

參考用書：Cormen 等，*Introduction to Algorithms*, 3rd Edition，Chapters 1–3；求和參閱 Appendix A。

本章的學習順序

部分	要學會回答的問題	共同案例
一、規格與證明	什麼算正確？為什麼一定停？	第一個匹配的線性搜尋
二、逐步計數	每輪實際做了幾次操作？	插入排序與逆序對
三、漸近界限	上界、下界、緊界的量詞有何不同？	具體常數與振盪反例
四、比較成長	定義、極限、對數法各何時適用？	對數、冪次、指數、階乘
五、級數與迴圈	範圍相依時，如何建立一般求和？	三角形、倍增、調和級數
六、模型與延伸	空間、位元長度與近似有哪些限制？	存活區間、溢位、誤差界

先完成主線，再讀延伸證明；遇到不熟悉的符號，可回到定義頁核對條件，而不是只記成長階表。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

先寫規格，再看程式

問題由合法輸入集合 D 與輸出關係 $R(x)$ 描述。輸入 $x \in D$ 可以有不只一個合法答案；演算法則是產生答案的明確程序。

排序的輸入是一列資料，每筆有比較用的鍵值與固定身分。例如 $(3, a), (1, b), (3, c)$ 中，a, b, c 用來辨識原資料，不是額外排序依據。

輸出要求	檢查方式
鍵值非遞減	相鄰鍵值皆滿足 $A[i].key \leq A[i+1].key$
不遺失、不複製資料	輸出是輸入資料的排列，重複次數也相同
若要求穩定排序	相等鍵值維持原來的先後

只有「有序」仍不夠：刪光資料後的空序列雖有序，卻不是原輸入的排序結果。

重複值要靠身分辨認

輸入 [3a, 1b, 3c] → 鍵值排序；字母表示資料身分

有序、保存、穩定

1b 3a 3c

三項要求全部成立

有序、保存

1b 3c 3a

等鍵 a、c 的先後反轉

有序，但遺失資料

1b 3a

少了 3c，不是排序結果

穩定性是額外要求。若題目只要求有序且保存全部資料，交換兩筆相等鍵值不會使答案錯誤；若要求穩定，就必須保留 a 在 c 前面。

空輸入與單筆輸入都已排序；演算法仍須合法處理，不能讀取不存在的第一筆。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

正確結果與一定停下，是兩件事

部分正確性（partial correctness）：只要演算法終止，輸出就符合規格。完全正確性（total correctness）還要求每個合法輸入都會終止。

```
while true
    skip
```

這段程式從不交出錯誤答案，也從不交出答案。若只證「終止後正確」，還不能說它完成任務。

證明迴圈會停，常找一個非負整數量：每次迭代嚴格下降，且不會降到負數。有限行程式、有限測試案例，都不能代替這項論證。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

線性搜尋：先固定返回哪一個

給陣列 $A[1..n]$ 與目標 x，返回**第一個**符合 $A[i] = x$ 的索引；若沒有則返回 NIL。允許 $n = 0$，比較與讀取一筆資料各為常數成本。

```
FIRST-MATCH(A, x)
    i = 1
    while i <= n
        if A[i] == x: return i
        i = i + 1
    return NIL
```

本章陣列索引從 1 開始；$A[a..b]$ 在 $a > b$ 時表示空區間，不會讀取元素。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

陣列參照與資料身分，要分開理解

本章偽碼以參照存取陣列。把陣列傳給程序後，修改 $A[i]$ 會改到呼叫端看見的資料；只把程序內的變數重新綁到另一個陣列，則不會改綁呼叫端的變數。

操作	結果
$A = [3, 1]$；令 $B = A$；再令 $B[1] = 8$	兩個名稱指向同一陣列，A 也變成 $[8, 1]$
$A = [3, 1]$；令 $B = copy(A)$；再令 $B[1] = 8$	陣列儲存體不同，A 仍是 $[3, 1]$

陣列複製若只複製物件參照，兩個陣列仍可能指向同一批物件；「陣列獨立」不表示「每個物件都深層複製」。排序中標記的身分 a, b, c 則用來辨識資料，不能直接當作其記憶體位址。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

搜尋的不變量，記錄已排除的範圍

每次判斷 `i <= n` 前，維持：$1 \le i \le n+1$，而 $A[1..i-1]$ 都不等於 x。

階段	理由
初始化	$i = 1$，已排除範圍為空，條件成立
維護	若本筆不等於 x，令 $i \leftarrow i + 1$；剛檢查的位置加入排除範圍
找到時	$A[i] = x$，且之前沒有匹配，所以返回的是第一個
掃完時	$i = n + 1$，所有位置都已排除，因此返回 NIL

未返回的每次迭代都讓非負整數 $n - i + 1$ 減 1，因此必停。$n = 0$ 時直接返回 NIL，不讀取 $A[1]$。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

i 之前已排除，i 之後還不能下結論

A = [6, 2, 6, 5]；目標 x = 5



綠底：已證明不匹配。青框：目前位置。白底：尚未檢查。

在 $A = [6, 2, 6, 5]$ 搜尋 5。到 $i = 3$ 前，只能斷言前兩筆不匹配；即使看見第 3 筆是 6，也要實際比較後才能擴大排除範圍。

到 $i = 4$ 時找到 5，而前段已證明沒有 5，所以返回最早匹配。圖中的「尚未檢查」不等於「不匹配」。

同一段程式，三種完整執行

輸入與目標	依序比較的位置	返回值	鍵值比較次數
$[4, 2, 4]$，找 4	1	1	1
$[4, 2, 4]$，找 7	1、2、3	NIL	3
空陣列，找 4	無	NIL	0

第一列同時驗證「最早匹配」與提前返回；第二列走完迴圈；第三列檢查空輸入。這些測試能找錯，但一般正確性仍來自上一頁的不變量。

若成功位置均勻分布在 $1..n$，平均比較數為 $(n+1)/2$；若輸入分布不同，不能沿用這個數字。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

計時之前，先說明在數什麼

本章先採 word-RAM 模型：索引與資料可放入固定數量的機器字；讀寫、比較與模型允許的基本算術各計常數成本。這是分析的約定，不是所有實際運算都一樣快。

觀察對象	常見成本單位
陣列搜尋	鍵值比較次數、讀取次數
排序	比較次數、搬移次數、額外空間
大整數演算法	位元數與多字運算成本
外部儲存	區塊傳輸次數

若整數含 b 個位元，任意精度乘法不能不加說明就視為一次常數操作。後面的結論都要連同成本模型閱讀。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

輸入大小不一定是數值本身

正整數 N 的二進位長度是 $b = \lfloor \lg N \rfloor + 1$，其中 $\lg = \log_2$。以 N 為次數的迴圈，對輸入長度 b 可能是指數成本。

例如 $N = 13$ 寫成 `1101`，只需 4 個位元。若逐一試到 N，迭代數為 $\Theta(N)$；對 b 位數輸入，$2^{b-1} \le N < 2^b$，所以是 $\Theta(2^b)$ 次。

陣列通常以元素數 n 為規模；圖常同時用頂點數 V、邊數 E。把二參數成本寫成 $O(V + E)$，比默默把所有東西叫作 n 更清楚。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

最好、最壞、平均：先選成本函數

對長度為 n 的輸入集合 D_n，令 $C(x)$ 為該輸入的成本。

$$B(n) = \min_{x \in D_n} C(x), \qquad W(n) = \max_{x \in D_n} C(x).$$

平均成本還需要指定機率分布 P_n：

$$A(n) = \mathbb{E}_{x \sim P_n}[C(x)] = \sum_{x \in D_n} P_n(x) C(x).$$

Big-O 並不等於最壞情況。 B, W, A 都是函數，都能各自求上界或下界。隨機演算法還要區分輸入的隨機性與演算法自己擲出的隨機位元。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

插入排序：把下一筆放進已排序區

輸入 $[(7,a),(2,b),(5,c),(2,d),(1,e)]$，簡記為 $[7a,2b,5c,2d,1e]$。每輪把一筆資料暫存為 key，只把左邊較大的資料往右搬。

```
INSERTION-SORT(A)
    for j = 2 to n
        key = A[j]
        i = j - 1
        while i >= 1 and A[i].key > key.key
            A[i+1] = A[i]
            i = i - 1
        A[i+1] = key
```

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

and 由左至右短路求值；$i = 0$ 時不讀取 $A[i]$。$n \leq 1$ 時外層不執行。比較採嚴格大於，讓舊的相等鍵留在新資料前面。

每一輪都留下可以核對的結果

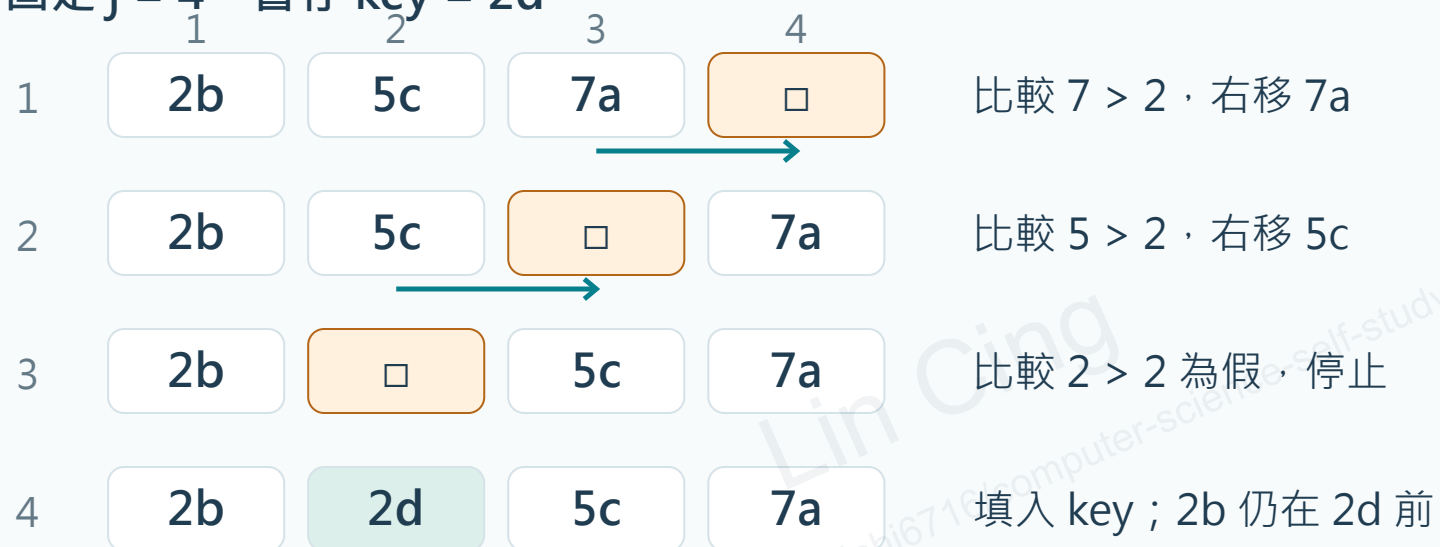
j／暫存資料	由右向左的鍵值比較	右移數 h_j	填入後的陣列
2／$2b$	$7 > 2$；到邊界	1	$2b, 7a, 5c, 2d, 1e$
3／$5c$	$7 > 5$、$2 > 5$ 為假	1	$2b, 5c, 7a, 2d, 1e$
4／$2d$	$7 > 2, 5 > 2$、$2 > 2$ 為假	2	$2b, 2d, 5c, 7a, 1e$
5／$1e$	$7, 5, 2d, 2b$ 都大於 1	4	$1e, 2b, 2d, 5c, 7a$

最後 $2b$ 仍在 $2d$ 前面。共右移 8 次，但鍵值比較是 10 次：第 3、4 輪各多一次失敗比較；到 $i = 0$ 的第 2、5 輪不再比較鍵值。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

搬移時的重複值，用「洞」來理解

固定 j = 4，暫存 key = 2d



□ 是邏輯洞，不是寫入的新值；key 在填回前一直由暫存變數保存。

空格是**邏輯洞**，不是寫入的空值。每步由「洞外位置＋暫存 key」保存原資料；第一次搬移後，實體陣列有兩份 $7a$，所以不能把整段實體陣列當成原輸入的排列。

外層證明：有序、保存、穩定一起維持

令 S 為開始排序前的輸入副本。每輪開始時：$A[1..j-1]$ 是 $S[1..j-1]$ 的穩定排序，且 $A[j..n] = S[j..n]$。

階段	不變量如何成立
初始化 $j = 2$	單筆前段已排序，後段未改動
維護	右移所有比新鍵大的尾段，再填入新資料；等鍵不跨越，後段不變
終止 $j = n + 1$	前段就是全部輸入，故全部已穩定排序

內層的 i 每次減 1，最多搬 $j - 1$ 筆；外層最多 $n - 1$ 輪，所以必定終止。$n \leq 1$ 另外由不執行迴圈成立。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

內層證明：洞的位置也要寫進條件

固定一輪 j，令 $B = A[1..j]$ 為該輪開始的副本，$B[1..j-1]$ 已排序。暫存 $key = B[j]$，令洞在 $i+1$。

每次內層判斷前：

1. 左段 $A[1..i] = B[1..i]$。
2. 右段 $A[i+2..j] = B[i+1..j-1]$，而這些鍵都大於 key。
3. 不計洞內舊值，再加上 key，恰好保存 B 的每筆資料。

初始化右段為空；搬移 $A[i]$ 到洞再令 $i \leftarrow i-1$，三項仍成立。停止時左段皆不大於 key，右段皆大於它，填洞便得到有序且保存資料的前段。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

把比較、搬移與迴圈判斷分開計數

對 $n \geq 2$，第 j 輪右移 h_j 次；若停在某個 $i \geq 1$ 的失敗鍵值比較，令 $q_j = 1$，否則令 $q_j = 0$。

計數對象	第 j 輪	全程
右移寫入	h_j	$\sum_{j=2}^{n} h_j$
最後填入 key 的寫入	1	$n - 1$
內層完整 guard 判斷	$h_j + 1$	$\sum h_j + n - 1$
鍵值比較	$h_j + q_j$	$\sum h_j + \sum q_j$

上例 $\sum h_j = 8, \sum q_j = 2$，所以比較 10 次、guard 判斷 12 次、陣列寫入 12 次。暫存 key 的讀取不算在「陣列寫入」裡。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

為什麼右移數恰好等於逆序對數？

逆序對（inversion）是原輸入中滿足 $p < q$ 且 $S[p].key > S[q].key$ 的位置對。令總數為 $I(S)$。

插入原來第 j 筆時，左側是前 $j-1$ 筆的排序；被右移的恰是其中鍵值大於 $S[j]$ 的資料。因此本輪 h_j 等於以 j 為右端點的逆序對數。

每個逆序對有唯一右端點，所以

$$\sum_{j=2}^{n} h_j = I(S), \qquad T(S) = \Theta(n + I(S)).$$

等鍵不算逆序。上例依右端點分別有 $1, 1, 2, 4$ 對，共 8 對，與右移表一致。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

同一演算法的最好、最壞與平均

輸入條件	逆序對數	時間成本
已非遞減排序	0	$\Theta(n)$
相異鍵嚴格遞減	$n(n-1)/2$	$\Theta(n^2)$
相異鍵的所有排列等機率	期望 $n(n-1)/4$	期望 $\Theta(n^2)$

平均情況的理由：對每對位置 $p<q$，令指示變數 X_{pq} 表示是否形成逆序。在均勻排列模型下，$\mathbb{E}[X_{pq}]=1/2$；線性期望給

$$\mathbb{E}[I]=\sum_{p<q}\mathbb{E}[X_{pq}]=\binom{n}{2}\frac{1}{2}.$$

不需要假設各 X_{pq} 獨立。含重複鍵或非均勻輸入時，必須重新檢查分布。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

漸近分析：忽略什麼，保留什麼？

我們關心 $n \to \infty$ 時的成長階，允許忽略固定常數倍率與有限多個小輸入。這不代表小輸入的正確性可以忽略。

以下 n 取正整數，f 最終非負，g 最終嚴格正。$O(g), \Omega(g), \Theta(g)$ 都是**函數集合**；$f = O(g)$ 是常用寫法，意思是 $f \in O(g)$。

例如 $n \in O(n^2)$ 正確，只是上界不緊。若要表示已找到上下同階的界，應寫 Θ，不能把所有 Big-O 都讀成「恰好」。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

上界、下界與緊界的量詞

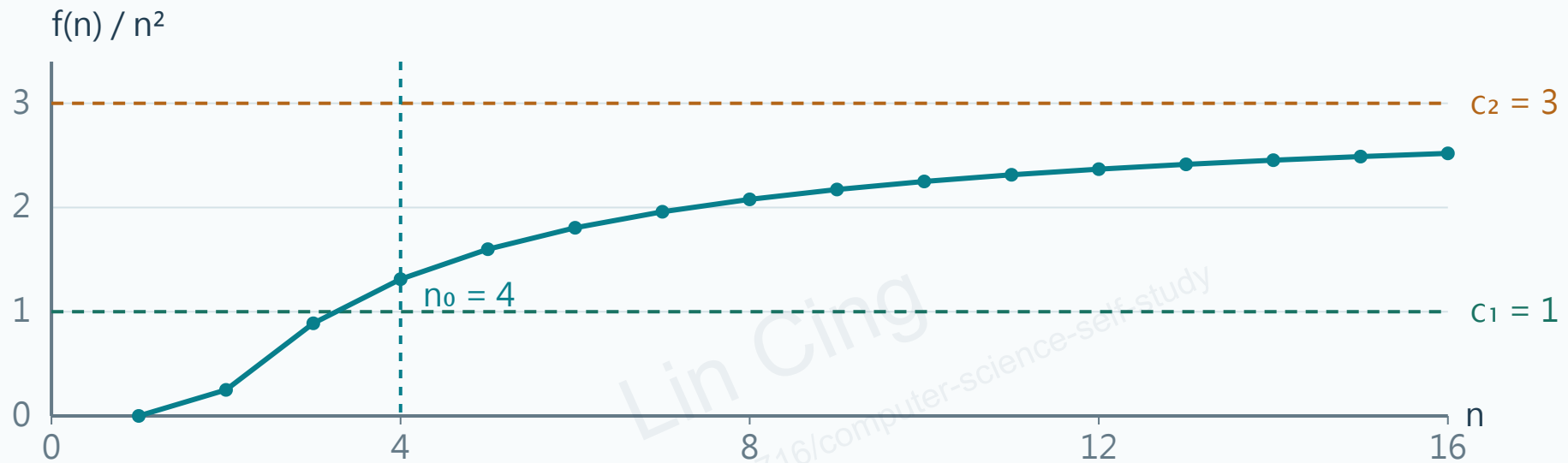
記號	正式條件：對所有整數 $n \geq n_0$
$f \in O(g)$	存在 $c > 0, n_0$，使 $0 \leq f(n) \leq cg(n)$
$f \in \Omega(g)$	存在 $c > 0, n_0$，使 $0 \leq cg(n) \leq f(n)$
$f \in \Theta(g)$	存在 $c_1, c_2 > 0, n_0$，使 $0 \leq c_1 g(n) \leq f(n) \leq c_2 g(n)$

常數可以依固定的 f, g 而定，**不能依輸入大小 n 改變**。門檻不必最小；只要整段尾端都滿足條件即可。

要否定 $f \in O(g)$，必須對任意候選 c, n_0，都找得到 $n \geq n_0$ 使 $f(n) > cg(n)$；打敗一組猜測常數還不夠。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

把常數與門檻畫在同一張圖



n ≥ 4 時整段比值落在 1 與 3 之間；不要求 n₀ 以前也成立。

取 $f(n) = 3n^2 - 8n + 5$，$n \geq 4$ 時，$f(n)/n^2 = 3 - 8/n + 5/n^2 \geq 1$；$n \geq 1$ 時 $f(n) \leq 3n^2$。

因此 $c_1 = 1, c_2 = 3, n_0 = 4$ 可證 $f \in \Theta(n^2)$。圖顯示比值的位置；涵蓋所有 $n \geq 4$ 的理由是上述不等式，不是有限採樣點。

同一個函數，怎麼證明不是線性上界？

已知 $f(n) = 3n^2 - 8n + 5 \geq n^2$ 對 $n \geq 4$ 成立。假設它也在 $O(n)$，則存在固定 c, n_0，使 $f(n) \leq cn$ 對所有 $n \geq n_0$ 成立。

選整數 $n > \max\{4, n_0, c\}$，便得到

$$n^2 \leq f(n) \leq cn \quad \Longrightarrow \quad n \leq c,$$

與 $n > c$ 矛盾。因此 $f \notin O(n)$。

這種證明的重點是：先讓對方任選常數，再選足夠大的輸入。不是先固定一個 n，再觀察某組常數不適用。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

兩側的門檻不同，取較大的那一個

若 $f \in O(g)$ 的證據是 (c_U, N_U)，$f \in \Omega(g)$ 的證據是 (c_L, N_L)，取 $n_0 = \max(N_U, N_L)$，兩側就同時成立。因此

$$\Theta(g) = O(g) \cap \Omega(g).$$

其他常用推論也要保留條件：

性質	理由或條件
O 的遞移性	兩個倍率相乘、兩個門檻取最大
$f \in O(g) \Leftrightarrow g \in \Omega(f)$	把同一不等式反向解讀，倍率取倒數
$f + g = \Theta(\max(f, g))$	非負時 $\max(f, g) \le f + g \le 2\max(f, g)$

若允許正負相消，最後一項不能直接套用，例如 $n + (-n) = 0$。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

little-o：每個倍率都能壓過它

$$f \in o(g) \iff \forall c > 0 \; \exists n_0(c) \; \forall n \geq n_0(c) : \; 0 \leq f(n) < cg(n).$$

Big-O 只要求找到一個倍率；little-o 要求**任何正倍率**最終都成立。例如 $7n \in o(n^2)$：給任意 $c > 0$，取整數 $n_0 > 7/c$ 即可。

同理，$f \in \omega(g)$ 要求對每個 $c > 0$，最終都有 $0 \leq cg(n) < f(n)$。因此 $n^2 \in \omega(7n)$。

在最終為正的條件下，$f \in o(g)$ 等價於 $f/g \to 0$，$f \in \omega(g)$ 等價於 $f/g \to \infty$。$7n^2$ 與 n^2 同階，不是 little-o。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

比值極限是工具，不是唯一判準

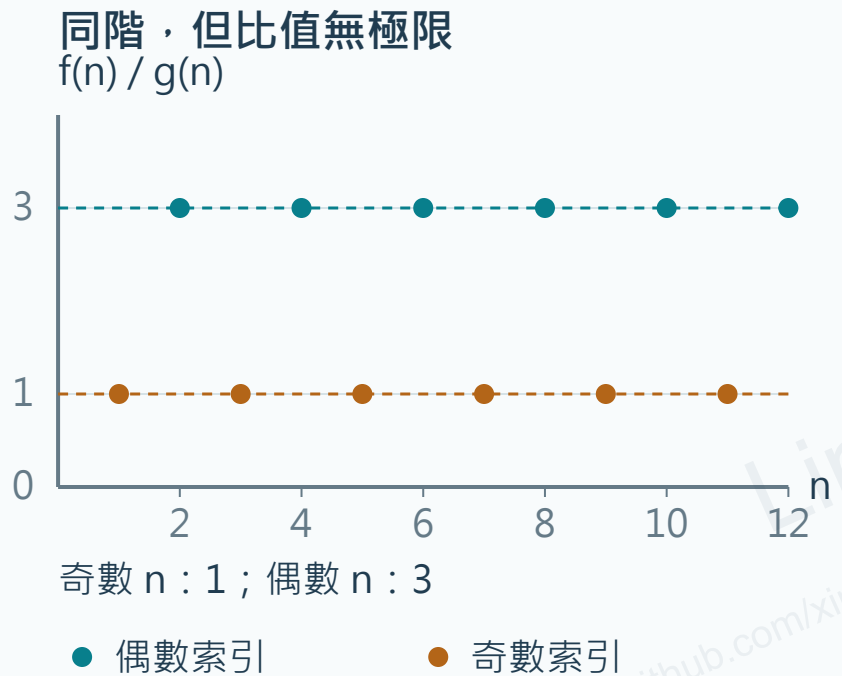
$\lim f(n)/g(n)$	可以得到的結論
0	$f \in o(g)$
有限正數 L	$f \in \Theta(g)$
$+\infty$	$f \in \omega(g)$
不存在	必須繼續分析，不能直接判定不可比較

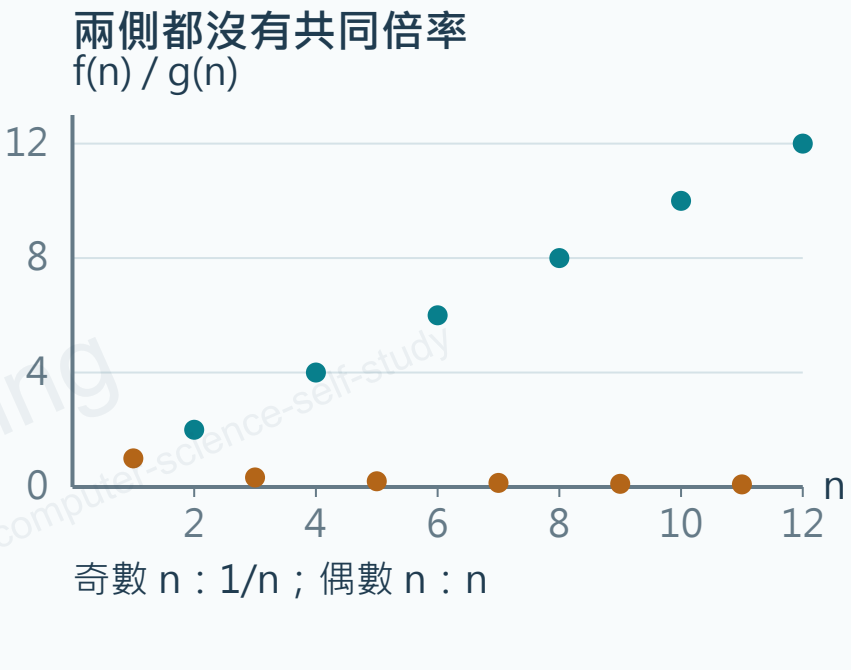
有限正 L 的理由：取 $\varepsilon = L/2$，最終有 $L/2 < f/g < 3L/2$，所以兩側都能用固定倍率夾住。

只有比值存在時才使用前三列。對數法也需檢查定義域；$\lg f / \lg g \to 1$ 本身**不保證** $f = \Theta(g)$。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

沒有極限，也可能同階





左圖 $f = n(2 + (-1)^n), g = n$：$n \leq f \leq 3n$，比值雖無極限仍同階。右圖偶數取 $f = n^2$、奇數取 $f = 1$，$g = n$：偶數比值無界，否定 $O(g)$；奇數比值趨 0，否定 $\Omega(g)$。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

函數界與輸入界，不要交換量詞

令 $C(x)$ 為輸入成本，$W(n) = \max_{x \in D_n} C(x)$。若 $W(n) \in O(n^2)$，表示存在**共同**常數 c, n_0，對所有夠大的 n、所有 $x \in D_n$，都有 $C(x) \le cn^2$。

若只說「對每個輸入都找得到一個常數」，而常數可以隨輸入增大，便不能得到共同上界。

插入排序的 $W(n) = \Theta(n^2)$，只保證最壞輸入達平方階；已排序輸入仍是 $\Theta(n)$。因此不能推成「每個輸入都至少需要平方時間」。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

函數比較的三種實際做法

方法	適合的情況	要避免的跳步
定義夾界	多項式、振盪、要給常數	只列數值或只證一側
比值極限	正函數且極限可求	把極限不存在當成無法比較
取對數比較	正的乘積、冪次、指數	把對數同階誤當原函數同階

例如 $f = n \lg n, g = n$，雖然 $\lg f / \lg g = 1 + \lg\lg n / \lg n \to 1$，但 $f/g = \lg n \to \infty$，所以 $f \in \omega(g)$。

對數能協助比較大小；是否達到常數倍率的同階，仍須回到原函數或更精確的對數差。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

成長階梯上的常數，都必須固定

對固定 $a > 1$、$b > 0$、$\varepsilon > 0$、$k > 0$，有

$$1 \prec \lg\lg n \prec (\lg n)^b \prec n^{\varepsilon} \prec a^n \prec n! \prec n^n,$$

其中 $f \prec g$ 表示 $f = o(g)$；只比較充分大的正整數 n。

取對數令 $x = \ln n$，則 $(\lg n)^b / n^{\varepsilon}$ 等於固定常數乘 $x^b / e^{\varepsilon x}$，趨近 0。又 $\ln(n^k / a^n) = k \ln n - n \ln a \to -\infty$，故 $n^k = o(a^n)$。

這裡使用基本極限 $\ln n / n \to 0$，下一頁補上理由。若底數或指數依 n 變動，不能套用「固定常數」的結論。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

為什麼對數會被正次方壓過？

對實數 $x \geq 4$，$\sqrt{x} - \ln x$ 的導數為 $(\sqrt{x} - 2)/(2x) \geq 0$，且在 $x = 4$ 時為正。因此 $0 \leq \ln x / x \leq 1/\sqrt{x} \to 0$。

現在固定 $b, \varepsilon > 0$，令 $t = \ln n$，取自然對數比較：

$$\ln \frac{(\ln n)^b}{n^\varepsilon} = b \ln t - \varepsilon t = t \left(b \frac{\ln t}{t} - \varepsilon \right).$$

因 $\ln t / t \to 0$，括號最終不超過 $-\varepsilon/2$，整式趨向負無限，原比值便趨 0。這就證出 $(\ln n)^b = o(n^\varepsilon)$；換成固定底數只差常數。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

對數、指數與底數轉換

式子	正確解讀
$\log_a n = \lg n / \lg a$	固定 $a > 1$ 時，不同底數只差常數
$\lg(n^k) = k \lg n$	先做冪次再取對數
$\lg^k n = (\lg n)^k$	先取對數再做冪次
$2^{\lg n} = n$	底數與對數配對
$2^{2 \lg n} = n^2$	指數裡的常數不能丟掉

$\lg n$ 與 $2 \lg n$ 同階，卻不能推出 $2^{\lg n}$ 與 $2^{2 \lg n}$ 同階。對數底數若隨輸入改變，例如 $\log_n n = 1$，也不是固定底數的換底情況。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

從迴圈到級數：每層的範圍要重看

```
count = 0
for i = 1 to n
    for j = 1 to i
        count = count + 1
```

第 i 輪執行 i 次，因此總次數為

$$\sum_{i=1}^{n} i = \frac{n(n+1)}{2} = \Theta(n^2).$$

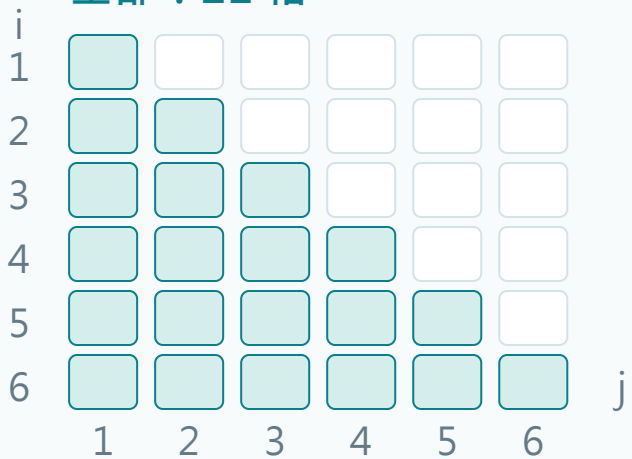
不能把內層「平均大約一半」當證明；精確求和已給一般公式。另一種夾界：總數不超過 n^2；後半至少 $n/2$ 輪，每輪至少 $n/2$ 次，所以至少 $n^2/4$。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

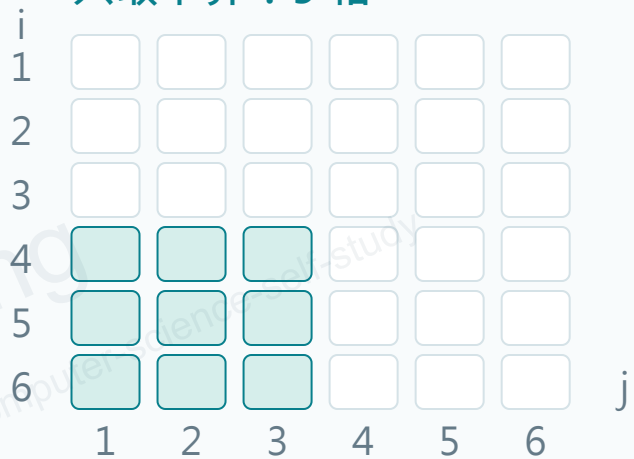
三角形不是少一個成長階

n = 6；每一格代表一次 visit(i, j)

全部：21 格



只取下界：9 格



上界：不超過 6 × 6 = 36 格。下界：第 4–6 輪，各取前 3 格。

每格代表一次內層執行。$n = 6$ 共 $1 + 2 + 3 + 4 + 5 + 6 = 21$ 次；第 4–6 輪各取前三次，已有 $3 \times 3 = 9$ 次。這是上一頁一般平方下界的具體子集合。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

倍增迴圈：數的是乘了幾次 2

```
i = 1
while i <= n
    visit(i)
    i = 2*i
```

若採無溢位的整數算術且 $n \geq 1$，依次造訪 $1, 2, 4, \ldots, 2^{\lfloor \lg n \rfloor}$，共 $\lfloor \lg n \rfloor + 1$ 次；$n = 13$ 時是 $1, 2, 4, 8$，共 4 次。

固定寬度實作必須避免 `2*i` 溢位。例如本輪造訪後，若 $i > \lfloor n/2 \rfloor$ 就停止，否則才倍增。數學偽碼的終止證明不能直接涵蓋溢位後的機器行為。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

幾何級數：最末一層已決定同階

若外層 $i = 1, 2, 4, \ldots, 2^k \le n$，內層各做 i 次，則總成本

$$1 + 2 + 4 + \cdots + 2^k = 2^{k+1} - 1.$$

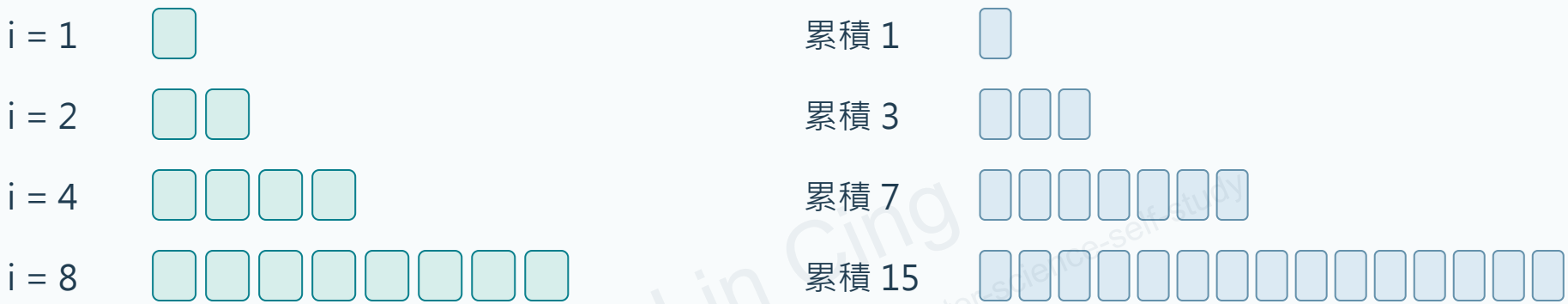
因 $2^k \le n < 2^{k+1}$，有 $n/2 < 2^k \le n$，所以總成本是 $\Theta(n)$，不是 $\Theta(n \lg n)$。

$n = 13$ 時四輪各做 $1, 2, 4, 8$ 次，共 15 次。雖有 $\Theta(\lg n)$ 輪，但不能把每輪都當成 n 次後，誤認為已得到緊界；那樣只證出寬鬆上界。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

倍增的是每輪工作量，不是輪數乘最後一輪

n = 13，最後造訪的二次冪是 8



每輪工作量：1、2、4、8

累積：1、3、7、15

左圖每格是一份工作，四輪各做 $1, 2, 4, 8$ 份；右圖的累積量是 $1, 3, 7, 15$。最後一輪的 8 份已占總量一半以上。

對一般最後一輪 2^k，$2^k \le \sum_{r=0}^{k} 2^r < 2 \cdot 2^k$。所以總量與最後一輪同階，而不是再多乘一個輪數。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

調和級數：用面積取得兩側界

令 $H_n = \sum_{i=1}^{n} 1/i$。$1/x$ 在正半軸遞減，因此

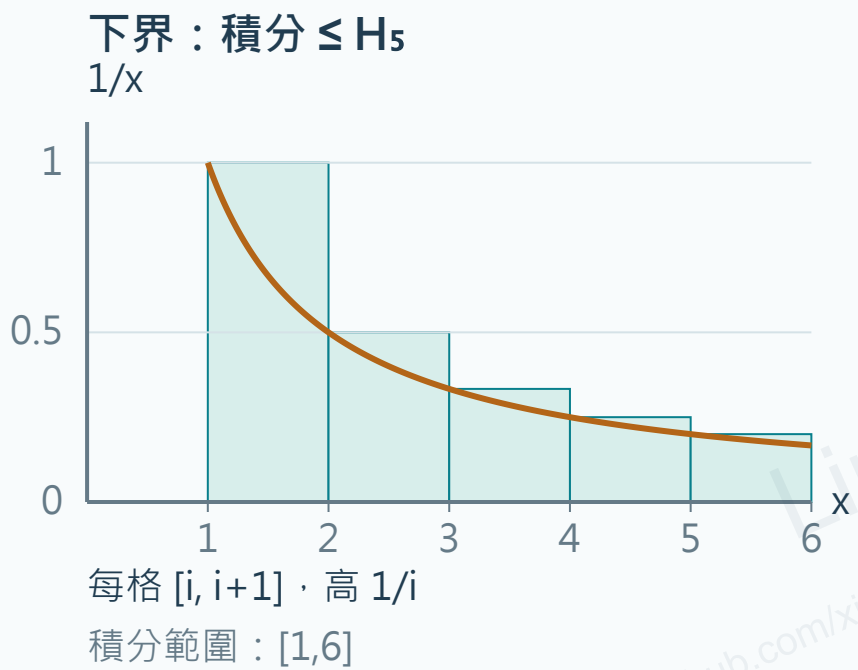
$$\int_{1}^{n+1} \frac{dx}{x} \le H_n \le 1 + \int_{1}^{n} \frac{dx}{x}.$$

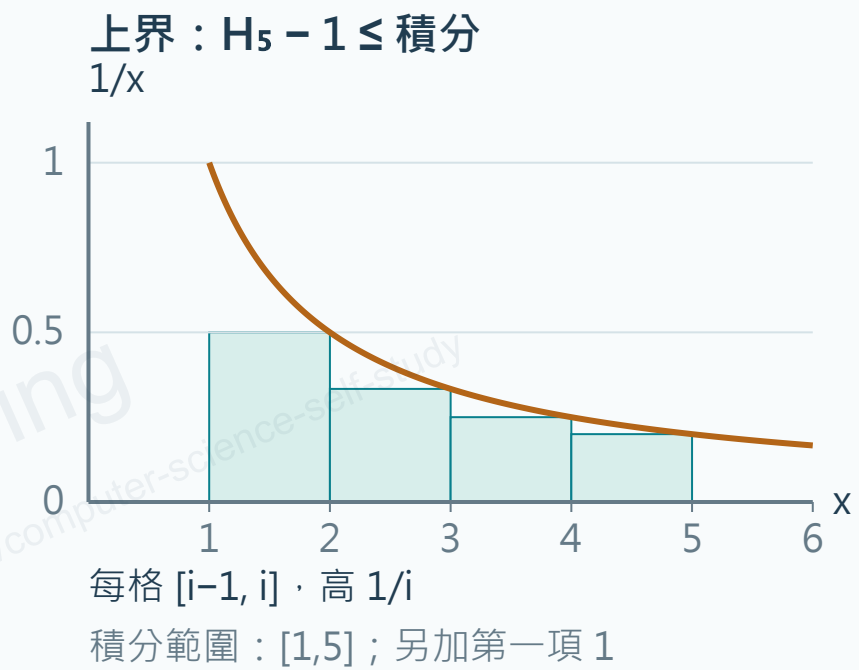
亦即 $\ln(n+1) \le H_n \le 1 + \ln n$。當 $n \ge 3$，$1 + \ln n \le 2 \ln n$，故 $H_n = \Theta(\ln n) = \Theta(\lg n)$。

上界把第一項 1 分開，再用每段左側的函數值蓋住第 2 項以後；下界則在每段用左端矩形蓋住積分。下一頁把兩種矩形分開畫。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

同一條曲線，兩種矩形方向





左圖 $n = 5$：每段 $[i, i+1]$ 的矩形高 $1/i$，總面積 H_5 大於 $\int_1^6 dx/x$。右圖第 $i = 2..5$ 項的矩形放在 $[i-1, i]$，位於曲線下，所以 $H_5 - 1 \le \int_1^5 dx/x$。

$H_5 = 137/60 \approx 2.2833$，介於 $\ln 6 \approx 1.7918$ 與 $1 + \ln 5 \approx 2.6094$ 之間。兩圖的座標與面積單位相同。

相依的巢狀迴圈，可能是 n log n

```
for i = 1 to n
    j = i
    while j <= n
        visit(i, j)
        j = j + i
```

內層造訪 $i, 2i, \ldots, \lfloor n/i \rfloor i$，所以

$$S(n) = \sum_{i=1}^{n} \lfloor n/i \rfloor .$$

對 $x \geq 1$，$x/2 \leq \lfloor x \rfloor \leq x$，故 $nH_n/2 \leq S(n) \leq nH_n$，得到 $\Theta(n \lg n)$。$n = 6$ 時各輪是 $6, 3, 2, 1, 1, 1$ 次，共 14 次。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

冪次求和：不用展開也能找緊界

對固定實數 $p \ge 0$，考慮 $S_p(n) = \sum_{i=1}^{n} i^p$。上界直接用 n 項、每項不超過 n^p：$S_p(n) \le n^{p+1}$。

下界只取 $i = \lceil n/2 \rceil .. n$，至少 $n/2$ 項，每項至少 $(n/2)^p$，所以

$$S_p(n) \ge \frac{n}{2}\left(\frac{n}{2}\right)^p = \frac{n^{p+1}}{2^{p+1}}.$$

因此 $S_p(n) = \Theta(n^{p+1})$。這個證法保留了固定倍率，也說清楚為什麼 p 必須固定；$p = -1$ 是調和級數，不能套這個結論。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

階乘先取對數，再比較成長

對 $n \geq 4$，$n! \leq n^n$，因此 $\lg(n!) \leq n \lg n$。

另一方面，後半至少 $n/2$ 個因子都不小於 $n/2$，所以

$$\lg(n!) \geq \frac{n}{2} \lg(n/2) \geq \frac{n}{4} \lg n.$$

故 $\lg(n!) = \Theta(n \lg n)$。這不代表 $n! = \Theta(n^n)$：事實上兩者比值趨近 0。

對固定 $a > 1$，$\ln(n!/a^n) \geq (n/2)\ln(n/2) - n \ln a \to \infty$，可證 $a^n = o(n!)$。若需精確倍率，再使用 Stirling 公式，不能把對數裡的誤差隨意指數化。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

floor、ceiling 與餘數的邊界

對實數 x，$x-1<\lfloor x\rfloor\le x\le\lceil x\rceil<x+1$。因此整數 n 滿足

$$\lfloor n/2\rfloor+\lceil n/2\rceil=n.$$

對整數 a 與正整數 m，本章採數學餘數

$$a \bmod m=a-m\lfloor a/m\rfloor,\qquad 0\le a \bmod m<m.$$

例如 $-7 \bmod 3=2$，因為 $-7=3(-3)+2$。某些語言的 `%` 採截向零的商，會得到負餘數；把公式搬成程式時必須核對語義。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

連續整數分割，什麼時候能合併？

對 $x \geq 0$、正整數 a, b，有

$$\left\lfloor \frac{\lfloor x/a \rfloor}{b} \right\rfloor = \left\lfloor \frac{x}{ab} \right\rfloor, \qquad \left\lceil \frac{\lceil x/a \rceil}{b} \right\rceil = \left\lceil \frac{x}{ab} \right\rceil.$$

floor 的證明：令 $q = \lfloor x/(ab) \rfloor$，則 $abq \leq x < ab(q+1)$。除以 a 再取 floor，得 $bq \leq \lfloor x/a \rfloor \leq bq + b - 1$；再除以 b 取 floor，結果就是 q。

例如 $x = 17, a = 3, b = 2$，兩次 floor 是 $17 \to 5 \to 2$，直接除以 6 取 floor 也是 2。a, b 為正整數是上述整數區間論證的前提。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

ceiling 的端點，要改用嚴格下界

令 $q = \lceil x/(ab) \rceil$，便有 $ab(q-1) < x \le abq$。除以 a 再取 ceiling，得到

$$b(q-1) + 1 \le \lceil x/a \rceil \le bq.$$

再除以 b 取 ceiling，結果也是 q。$x = 17, a = 3, b = 2$ 時，$17 \to 6 \to 3$，與 $\lceil 17/6 \rceil = 3$ 一致。

對正整數 u, v，另有 $\lceil u/v \rceil \le (u+v-1)/v$、$\lfloor u/v \rfloor \ge (u-v+1)/v$：寫成 $u = qv + r, 0 \le r < v$，分 $r = 0$ 與 $r > 0$ 即可核對。實作 `(u+v-1)/v` 時仍須避免加法溢位。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

空間成本：同時活著多少，才是峰值

額外空間不含輸入本身，但通常包括暫存陣列與遞迴呼叫堆疊；若另排除輸出，須明寫。

情境	累積配置量	同時存活的峰值
依次配置並釋放 n 個單字區塊	$\Theta(n)$	$\Theta(1)$
配置 n 個單字且全部保留	$\Theta(n)$	$\Theta(n)$
共 K 次呼叫，峰值有 d 個常數大小框架同時存活	$\Theta(K)$	$\Theta(d)$

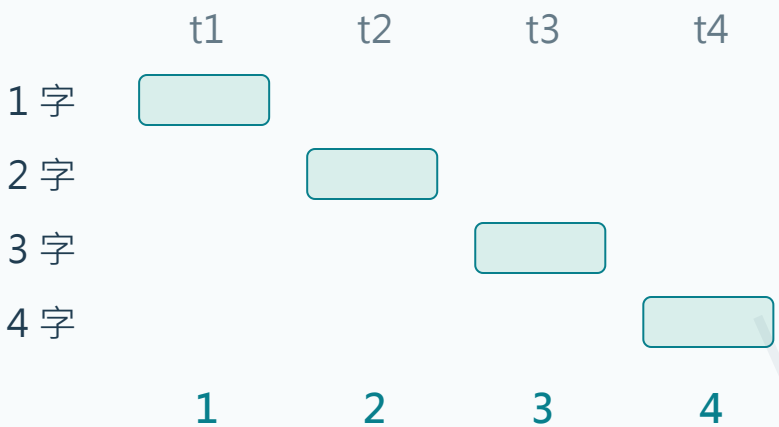
插入排序只用固定數量的索引與暫存資料，在單筆資料可用常數個字表示的假設下，額外空間為 $\Theta(1)$。若每筆是大型物件，還要說明搬的是物件還是參照。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

配過多少與同時保留多少：畫成存活區間

配置大小：1、2、3、4 個字；橫向代表四個處理時段

處理完立即釋放



峰值 4；累積配置 10

全部保留到最後



峰值 10；累積配置 10

依次配置 1、2、3、4 個字：立即釋放時峰值 4，全部保留時峰值 10；兩者累積配置量同為 10。只計圖中存活區塊，不含配置器碎片、快取或作業系統常駐記憶體。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

常數與交會點：成長階不能回答全部

同一成本單位下，兩種方法的精確模型是 $T_A(n) = 8n^2$、$T_B(n) = 64n \lg n$，$n \geq 2$。A 較快等價於 $n < 8 \lg n$。

n	$8n^2$	$64n \lg n$	比較
16	2048	4096	A 較快
32	8192	10240	A 較快
64	32768	24576	B 較快

對實數 $n \geq 4$，$n / \lg n$ 嚴格遞增；檢查 $n = 43, 44$ 分別落在兩側，可知正整數 $n \geq 2$ 的切換點在 43 與 44 之間。若模型來自量測，這個結論也只適用於該模型與環境。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

同階也可能差很多，快硬體也有單位

若 A 的速度是每秒 10^9 個模型步驟，B 是每秒 10^8 步，則應比較

$$t_A(n) = T_A(n)/10^9, \qquad t_B(n) = T_B(n)/10^8,$$

不能把步驟數與秒數直接混在同一個不等式。換硬體改變常數，換演算法可能改變成長階；兩者都需要明確成本模型。

空間上限也會限制可用方法。例如需要 n^2 個項目的表格，即使時間估算可接受，也可能放不進記憶體。實際選擇須同時檢查時間、空間與資料特性。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

綜合分析：從程式走到一般結論

分析一段新程式時，可以依下列順序建立答案，而不是先猜一個 Big-O：

1. 固定合法輸入、輸入規模、比較與算術模型。
2. 用小輸入列完整狀態，找到每輪實際做的事。
3. 寫不變量、邊界與遞減量，處理正確性及終止性。
4. 將每次成本寫成一般求和，或建立遞迴式。
5. 分開證上界與下界，註明最好／最壞／平均的對象。
6. 算峰值空間，核對溢位、別名與分布等前提。

本章的插入排序得到 $T(S) = \Theta(n + I(S))$，比只寫最壞 $O(n^2)$ 更能說明輸入特性如何改變成本。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

延伸：多項式最高次項為什麼主導？

令 $p(n) = a_d n^d + \sum_{i<d} a_i n^i$，固定 $d \geq 1$、$a_d > 0$，並設 $M = \sum_{i<d} |a_i|$。

對 $n \geq 1$，低階項的絕對值總和不超過 $M n^{d-1}$。再取 $n \geq 2M/a_d$，便有

$$\frac{a_d}{2} n^d \leq p(n) \leq \frac{3a_d}{2} n^d.$$

因此 $p(n) = \Theta(n^d)$，即使低階係數有負數仍成立；若最高次係數為負，就不符合本章時間函數最終非負的條件。$d = 0, a_0 > 0$ 時另得 $\Theta(1)$。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

延伸：Big-O 式子的等號不能反推

式子 $n^2 + n = n^2 + O(n)$ 表示右側存在一個屬於 $O(n)$ 的函數，使等式成立；本例選 $h(n) = n$ 即可。

由此可推出 $n^2 + n = \Theta(n^2)$，卻不能從 $f = \Theta(n^2)$ 反推 $f = n^2 + O(n)$：反例 $f = 2n^2$，差值 n^2 不在 $O(n)$。

若要逐項運算，先把匿名函數命名。例如 $u \in O(n), v \in O(n)$，用三角不等式得 $|u + v| \le (c_u + c_v)n$。若涉及相減，需允許帶符號的誤差界，不能忽略原來非負的定義慣例。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

延伸：e、ln 與近似的誤差

對 $x > -1$，$\ln(1+x) \le x$。因此正整數 n 有 $(1+1/n)^n \le e$。

更精確地，對 $x \ge 0$，

$$x - \frac{x^2}{2} \le \ln(1+x) \le x.$$

下界可由導數得到：$h(x) = \ln(1+x) - x + x^2/2$，$h(0) = 0$ 且 $h'(x) = x^2/(1+x) \ge 0$。代入 $x = 1/n$，得到

$$e^{1-1/(2n)} \le (1+1/n)^n \le e.$$

所以不只知道它趨向 e，也知道有限 n 時的一側誤差。近似式應附適用範圍，不能任意丟掉高階項。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

延伸：指數近似也能給明確餘項

Taylor 定理給 $e^x = 1 + x + (e^\xi/2)x^2$，其中 ξ 介於 0 與 x。因此 $|x| \le 1$ 時，

$$\frac{e^{-1}}{2}x^2 \le e^x - 1 - x \le \frac{e}{2}x^2.$$

所以 $e^x = 1 + x + \Theta(x^2)$ 的極限方向是 $x \to 0$，不是 $x \to \infty$。另由凸性，$e^x \ge 1 + x$ 對所有實數成立。

標準冪級數 $e^x = \sum_{k=0}^{\infty} x^k/k!$ 對所有實數收斂；若只取前幾項，還須像上式一樣控制餘項。本節使用基礎微積分的 Taylor 定理，不在此重新推導該定理。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

延伸：對數界也要涵蓋負的 x

對 $x > -1$，較一般的雙界為

$$\frac{x}{1+x} \le \ln(1+x) \le x.$$

上界令 $h = x - \ln(1+x)$，導數為 $x/(1+x)$，最小值在 0，且 $h(0) = 0$。下界令 $q = \ln(1+x) - x/(1+x)$，導數為 $x/(1+x)^2$，也在 0 取得最小值 0；兩個論證都涵蓋正負兩側。

對固定實數 z，充分大時代入 $x = z/n$，得 $z/(1+z/n) \le n\ln(1+z/n) \le z$。兩側都趨近 z，因此 $(1+z/n)^n \to e^z$。

當 $|x| < 1$，標準展開為 $\ln(1+x) = \sum_{k=1}^{\infty} (-1)^{k+1} x^k/k$；不能省略其適用範圍。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

延伸：Stirling 公式要保留什麼精度？

對正整數 n，Stirling 公式為

$$n! = \sqrt{2\pi n}(n/e)^n\big(1 + O(1/n)\big).$$

Stirling 定理給出主近似的相對誤差。取自然對數可得

$$\ln(n!) = n\ln n - n + \tfrac{1}{2}\ln(2\pi n) + O(1/n).$$

相對誤差 $O(1/n)$ 經取對數仍為 $O(1/n)$，因 $\ln(1+u) = O(u)$ 對充分小的 u 成立。反過來，若只知道對數相差 $O(n)$，指數化後可能相差指數倍率，不能說階乘值只有常數誤差。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

延伸：把 Stirling 誤差變成數值區間

Stirling 定理較精確的形式為：

$$n! = \sqrt{2\pi n}(n/e)^n e^{\alpha_n}, \qquad \frac{1}{12n+1} < \alpha_n < \frac{1}{12n} \quad (n \geq 1).$$

取 $n = 6$，主近似 $L = \sqrt{12\pi}(6/e)^6 \approx 710.078$，小於 $6! = 720$。代入誤差界得到

$$Le^{1/73} < 720 < Le^{1/72}.$$

修正項有明確範圍，不是任意四捨五入。由 $\alpha_n = \Theta(1/n)$ 與零附近 $e^u - 1 = \Theta(u)$，可得 $e^{\alpha_n} = 1 + \Theta(1/n)$。粗略乘積界只決定 $\lg(n!)$ 的成長階，這個定理則進一步控制主近似的相對誤差。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

延伸：反覆取對數，停止條件不能省

令 $\lg^{(0)} n = n$，$\lg^{(k+1)} n = \lg(\lg^{(k)} n)$；只在前一步大於 1 時再取對數。定義

$$\lg^* n = \min\{k \geq 0 : \lg^{(k)} n \leq 1\}.$$

輸入	依次得到的值	$\lg^* n$
1	1	0
2	$2 \to 1$	1
16	$16 \to 4 \to 2 \to 1$	3
65536	$65536 \to 16 \to 4 \to 2 \to 1$	4

$\lg^k n$ 是對數值的 k 次方；$\lg^{(k)} n$ 是函數反覆套用。兩者記號不同，成長率也不同。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

延伸：Fibonacci 的閉式與驗證

定義 $F_0 = 0, F_1 = 1, F_n = F_{n-1} + F_{n-2}$。設 $\varphi = (1+\sqrt{5})/2$、$\psi = (1-\sqrt{5})/2$，兩者都滿足 $r^2 = r + 1$。

因此 $G_n = (\varphi^n - \psi^n)/\sqrt{5}$ 滿足同一遞迴，且 $G_0 = 0, G_1 = 1$。依數學歸納法，$G_n = F_n$ 對每個非負整數成立。

因 $|\psi| < 1 < \varphi$，有 $F_n = \Theta(\varphi^n)$。但 F_n 的**數值大小**不等於計算它的演算法時間：樸素遞迴、表格法與快速倍增必須各自分析，後續遞迴章再比較。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

參考資料與署名

Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. Introduction to Algorithms, 3rd Edition.

Lin Cing 原創教學整理。原創內容以 CC BY 4.0 授權；參考書籍不隨教材散布。

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study